Universidade de Brasília
Departamento de Ciência da Computação

Recursão

Professora: Eliza Gomes

E-mail: eliza.gomes@unb.br

Introdução

- Uma função pode chamar qualquer função, inclusive ela mesma;
- Uma função que chama ela mesma é chamada de <u>função recursiva</u>;
- É um método para resolver problemas que envolve quebrar o problema em subproblemas cada vez menores até atingir um problema simples o bastante, que possa ser resolvido trivialmente

Exemplo

- Como esvaziar um vaso contendo 3 flores?
  - ✓ Verificamos se o vaso está vazio
  - ✓ Se o vaso não está vazio, tiramos 1 flor
  - ✓ Agora tem que esvaziar um vaso contendo 2 flores
- Como esvaziar um vaso contendo 2 flores?
  - ✓ Verificamos se o vaso está vazio
  - ✓ Se o vaso não está vazio, tiramos 1 flor
  - ✓ Agora tem que esvaziar um vaso contendo 1 flor
- Como esvaziar um vaso contendo 1 flor?
  - ✓ Verificamos se o vaso está vazio
  - ✓ Se o vaso não está vazio, tiramos 1 flor
  - ✓ Agora tem que esvaziar um vaso contendo 0 flores
- Como esvaziar um vaso contendo 0 flores?
  - ✓ Verificamos se o vaso está vazio
  - ✓ Como ele está vazio, o processo termina

Exemplo

- Como esvaziar um vaso contendo N flores?
  - ✓ Primeiro verificamos se o vaso está vazio
  - ✓ Se o vaso não está vazio, tiramos 1 flor
  - ✓ Agora tem que esvaziar um vaso contendo N – 1 flores

Exemplo: função recursiva para esvaziar vaso de flores

```
01  void esvaziarVaso(int flores){
02    if(flores > 0){
03        //remove uma flor
04        esvaziarVaso(flores - 1);
05    }
06  }
```

Funcionamento

- A ideia básica da recursão é **dividir** e **conquistar**:
  - ✓ Divide-se um problema maior em um conjunto de problemas menores
  - ✓ Esses problemas menores são então resolvidos de forma independente
  - ✓ As soluções dos problemas menores são combinadas para gerar a solução final

- Regras básicas:
  - ✓ Um algoritmo recursivo deve possuir um **caso base** (base case)
  - ✓ Um algoritmo recursivo deve **modificar o seu estado** e se aproximar do caso base
  - ✓ Um algoritmo recursivo deve **chamar a si mesmo**, recursivamente

Exemplo: fatorial de um número

- O fatorial de N é igual a N multiplicado pelo fatorial de (N – 1);

$$N! = N * (N - 1)!$$

- O processo termina quando atingimos o número 0;

$$0! = 1$$

- Trata-se de uma definição circular do problema: é preciso saber o fatorial de um número para calcular o de outro;
- Esse processo continua até que se chegue a um caso mais simples e que é a base do cálculo do fatorial: o fatorial de 0.

Exemplo: fatorial de um número

Exemplo: função recursiva para cálculo do fatorial

```
01  int fatorial (int n){
02    if(n == 0)
03        return 1;
04    else
05        return n*fatorial(n-1);
06  }
```

Exemplo: fatorial de um número

- A primeira etapa do cálculo pode ser chamada de **caminho de ida da recursão:**
  - ✓ Onde as chamadas da função são executadas até chegar ao **caso base**.
- Quando a recursão para, é chegado o momento de fazer o **caminho de volta da recursão** que, basicamente, consiste em fazer o caminho inverso devolvendo o valor obtido para quem fez aquela chamada da função, até chegar à primeira chamada da função;

Exemplo: fatorial de um número

$4! = 4 * 3!$

$3! = 3 * 2!$

$2! = 2 * 1!$

$1! = 1 * 0!$

$0! = 1$

Ida

$1! = 1 * 1$

$2! = 2 * 1$

$3! = 3 * 2$

$4! = 4 * 6 = 24$

Volta

Exemplo: fatorial de um número

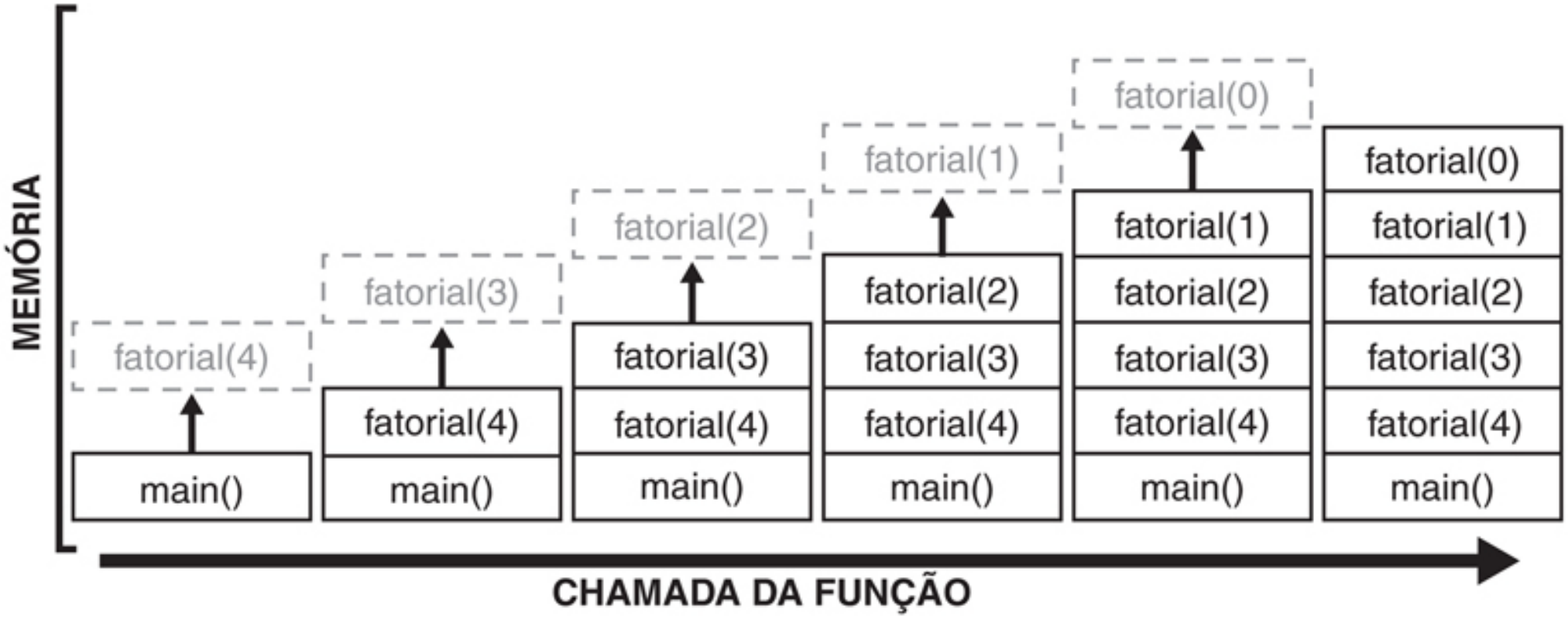
Exemplo: calculando o fatorial

```
01  #include <stdio.h>
02  #include <stdlib.h>
03  int fatorial (int n){
04    if(n == 0)
05        return 1;
06    else
07        return n*fatorial(n-1);
08  }
09  int main(){
10    int x;
11    x = fatorial(4);
12    printf("4! = %d\n",x);
13    system("pause");
14    return 0;
15  }
```

Exemplo: fatorial de um número

- Sempre que é feita uma chamada de função, seja ela qual for, o código atualmente em execução é pausado, e a função chamada é carregada na memória;
- A função **main()** é pausada quando chama a função **fatorial();**
- A função **fatorial()** também chama a si própria. Nesse caso, ela é pausada e carrega na memória uma cópia de si mesma, mas com valor de parâmetro diferente **(n – 1);**
- Esse processo se repete até que a função seja chamada com o valor de **n** igual a **0;**
- Esse é o **caso base** em que a função para de chamar a si mesma.

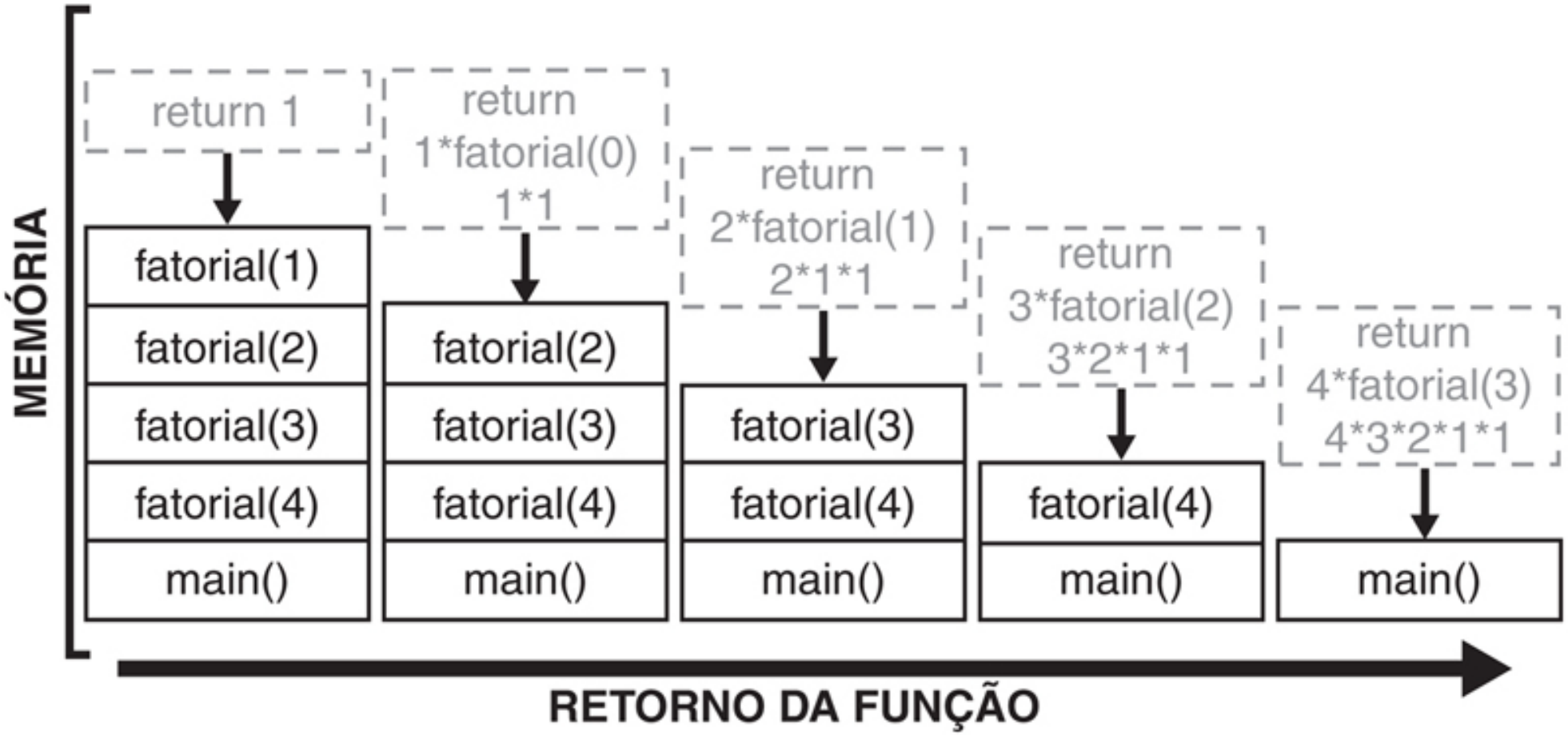
Exemplo: fatorial de um número



Exemplo: fatorial de um número

- Uma vez que chega ao **caso base**, é hora de fazer o caminho de volta da recursão;
- Sempre que uma função termina de executar, ela devolve (se existir) seu resultado para quem a chamou e, em seguida, é removida da memória;
- Assim:
  - ✓ A chamada da **função fatorial(0)** devolverá o valor **1** para a função à chamada da **função fatorial(1)**
  - ✓ A chamada da **função fatorial(1)** vai devolver o valor **1*1** para a chamada da **função fatorial(2)**, e assim por diante

Exemplo: fatorial de um número



Cuidados na implementação

- As formas recursivas dos algoritmos são consideradas *mais enxutas* e *mais elegantes* do que suas formas iterativas;
- Apresentam mais dificuldade na detecção de erros e podem ser ineficientes;
- Durante a implementação de uma função recursiva deve-se de ter em mente duas coisas:
  - ✓ **Critério de parada**: determina quando a função deve parar de chamar a si mesma. Se ele não existir, a função continuará executando até esgotar a memória do computador
  - ✓ **Parâmetro da chamada recursiva**: quando a função é chamada dentro dela mesma, deve-se sempre mudar o valor do parâmetro passado, de forma que a recursão chegue a um término. Se o valor do parâmetro for sempre o mesmo, a função continuará executando até esgotar a memória do computador.

Cuidados na implementação

- Tendem a necessitar de mais tempo e/ou espaço do que algoritmos iterativos;
- Sempre que é chamada uma função, é necessário espaço de memória para armazenar os parâmetros, as variáveis locais e o endereço de retorno da função;
- Em uma função recursiva, essas informações são armazenadas para cada chamada da recursão, sendo, portanto, a memória necessária para armazená-las proporcional ao número de chamadas da recursão.